SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. *Introduction:*

This section "AiyoDecision" is introducing the strategic solution of the identified problem statement highlighted in Product Review Documentation.

1.1. Purpose:

This system analysis document highlights the technical scope, architectural design, and LLM-integration decisions behind the development of AiyoDecision. The key elements covered include

1. ***Architecture**:*
    a. *A decoupled, serverless web application. The frontend is hosted on Vercel Edge networks, communicating with a lightweight Node.js microservice backend to ensure secure API key management*
2. ***Data Flows:***
    a. *How user financial constraints flow from the React frontend's localStorage, move through the backend API to be stitched into a structured prompt, and return as a parsed JSON dashboard update from the Gemini API.*
3. ***Model Process:***
    a. *The end-to-end workflow of the core feature: evaluating a user's financial decision against their constraints and rendering the quantifiable economic impact*
4. ***Role of Reference**:*
    a. *This document serves as the architectural blueprint for the development and QA teams to ensure the reliable implementation of AiyoDecision.*

*1.2. **Background**:*

Many students and young adults rely on manual, reactive tracking for their finances. They lack access to proactive, context-aware financial planning, leaving them vulnerable to poor cash flow management and digital fraud.

1. *Previous Version:*
    a. *Existing personal finance tools are static ledgers. They simply record what was spent. AiyoDecision introduces a major architectural shift by replacing static formulas with a Generative LLM service layer, shifting the paradigm from "Expense Tracking" to "Proactive Decision Intelligence."*
2. *Changes in Major Architectural Components*
3. *New capabilities that are only introduced in this*

© UMHackathon 2026

Email: umhackathon@um.edu.my

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
End-User (Citizens)	*Inputs financial constraints, queries the system for decision support, and monitors their AiyoDecision Score.*	*A simple, low-friction UI that provides instant, empathetic, and mathematically sound financial advice without requiring complex financial literacy.*
Development Team	*Builds the React frontend, configures the Node.js API, and engineers the Master Prompts.*	*Clear separation of concerns between the client UI and the LLM inference engine; modular codebase for rapid iteration.*
QA Team	*Validates the system behavior, ensuring the LLM does not hallucinate math and adheres strictly to JSON schemas.*	*Defined guardrails, predictable JSON response structures, and comprehensive coverage of edge cases (e.g., handling dangerous user inputs).*
Development Team	*Build the platform and maintain it*	*Clear API usage contracts, codebase is modular and well-documented architecture*
QA Team	*Validating the system behavior for all the user roles*	*Defined scope, the request and response formats, and coverage of edge cases*

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Progressive Web Application (PWA) optimized for mobile web browsers.*
Architecture	*Client-Server Serverless Architecture (React + Node.js Microservice).*

AiyoDecision is structured with a decoupled client-server model. The React frontend handles all user interactions and state management (utilizing localStorage for the MVP). The backend acts as a secure middleware layer that constructs the prompts and communicates with the Google Gemini API.

2.1.2 LLM as Service Layer

The Google Gemini 2.5 Flash model is integrated not as a chatbot, but as the core analytical engine.

a. Context Window & Prompt Construction: When a user triggers an action, the frontend stitches together a [USER FINANCIAL PROFILE] (static constraints) and [RECENT ACTIVITY] (dynamic inputs). This structured text is passed to the backend, which injects the "Master System Instructions" before calling the Gemini API.

b. Parsing & Routing: The Gemini API is strictly configured to output JSON. The backend receives this JSON, parses it, and passes it to the frontend to update the React state (rendering the Dashboard metrics or the Explainability Microcopy).

c. Token Limitation & Chunking: To prevent context bloat, historical transactions are not sent to the LLM. Only the user's current baseline and immediate query are passed, ensuring rapid inference and low token usage.

Fallback Behavior: If the LLM times out or returns a malformed JSON, the backend catches the error and returns a predefined safe state to the frontend (e.g., {"status": "error", "message": "*AiyoDecision* is recalibrating, please try again."}), ensuring the app does not crash.

© UMHackathon 2026

d. *Do demonstration of each key system features does operate from start to finish*

2.2. Technological Stack

2.2.1. Frontend React.js, Tailwind CSS, Framer Motion (Deployed on Vercel).

2.2.2. Backend Node.js, Express.js (RESTful API middleware deployed on Render/Cloud Run).

2.2.3. Database Browser localStorage (for rapid MVP prototyping of persistent user state).

2.2.4. Cloud/Deployment Google Gemini 2.5 Flash API (via Google AI Studio).

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)

a. *User Input: User submits a financial scenario (e.g., "Should I rent this condo?") via the React UI.*
b. *State Retrieval: The React app pulls the user's constraints and income from localStorage.*
c. *API Request: The client sends a POST request with the combined context to the Node.js backend.*
d. *LLM Inference: The backend formats the prompt and calls the Gemini API. Gemini processes the constraints, calculates the economic impact, and returns a structured JSON object.*
e. *UI Render: The frontend receives the JSON and dynamically updates the AiyoDecision Score and Impact Dashboard.*

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1. *Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026.* ***[e.g. in Table]***

#	*Features*	*Description*
1	*Progressive Onboarding*	*Users input baseline income, debt, and strict financial constraints (e.g., "Keep RM500 buffer") into a secure local profile.*

© UMHackathon 2026

2	*"Decide Lah!" Engine*	*An interactive query tool where users simulate financial decisions and receive mathematically justified approvals or rejections from the AI.*
3	*Dynamic Health Dashboard*	*A real-time display of the AiyoSafe Score, complete with context-aware microcopy explaining exactly why the score changed based on recent actions*
4	*Quantifiable Impact Tracker*	*Widgets displaying estimated RM saved, late fees avoided, and time saved by using the system.*

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	***Requirements (e.g.)***	***Implementation (e.g)***
Scalability	*Must handle sudden influxes of users querying the AI simultaneously.*	*Deployed on Vercel's Edge network for instant frontend delivery, with serverless backend scaling.*
Token Latency	*The LLM decision must return fast enough to maintain a seamless user experience.*	*Utilization of Gemini 2.5 Flash (optimized for speed) with strict JSON output limits. Loading states are implemented in the UI.*
Cost Efficiency	*Token costs must remain minimal per user session.*	*Implementation of a "Stitched Context" strategy. Only the condensed financial profile is sent, rather than thousands of database rows.*
Security	*API keys must not be exposed to the public.*	*API calls are routed entirely through the secure Node.js backend environment variables, never exposed in the React client.*

© UMHackathon 2026

3.3. Out of Scope / Future Enhancements

a. *Integration with Open Banking APIs (e.g., automated syncing with Maybank or CIMB accounts).*

b. *Direct OCR processing of physical receipts (currently scoped to text-based or multimodal AI extraction, but dedicated OCR pipelines are future enhancements).*

c. *Multi-user/Family account syncing for household budget optimization.*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. ***Prompt A/B Testing:*** *Iterative testing of the System Instructions to balance strict constraint adherence versus flexible financial advice.*

4.1.2. ***Emergency Rollback:*** *If the Gemini API experiences downtime, the system gracefully degrades, alerting users that live AI analysis is temporarily paused while retaining their localized dashboard data.*

4.2. Assumptions:

a. *Users will provide reasonably accurate baseline financial data during the onboarding phase.*

b. *Users have access to modern web browsers capable of supporting React and localStorage.*

4.3. ***External Dependencies:*** *Do list all the external tools that are applied to your system to function. [e.g. below]*

Tools	*Purpose*	*Risks*
Google Gemini 2.5 Flash	*Core LLM reasoning engine for parsing context, enforcing constraints, and generating the JSON output.*	*High Risk: Rate limits or API outages. Mitigation: Comprehensive try/catch error handling in the backend to prevent frontend crashes.*

© UMHackathon 2026

Vercel	*Edge hosting for the React frontend.*	*frontend. Low Risk: Highly reliable infrastructure.*

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. ***Project Timeline:*** *The MVP was developed within an accelerated hackathon sprint, prioritizing the LLM integration logic and UI/UX design over traditional database scaffolding.*

5.2. ***Team Members Role:*** *Ng Jen Wen as Fullstack Architecture & Multimedia UI Integration, Ang Dao Jun as Backend Architecture & API Routing Middleware, James Keo Lin Shun as Project Management, Documentation, and Pitch Deck Coordination.*

© UMHackathon 2026